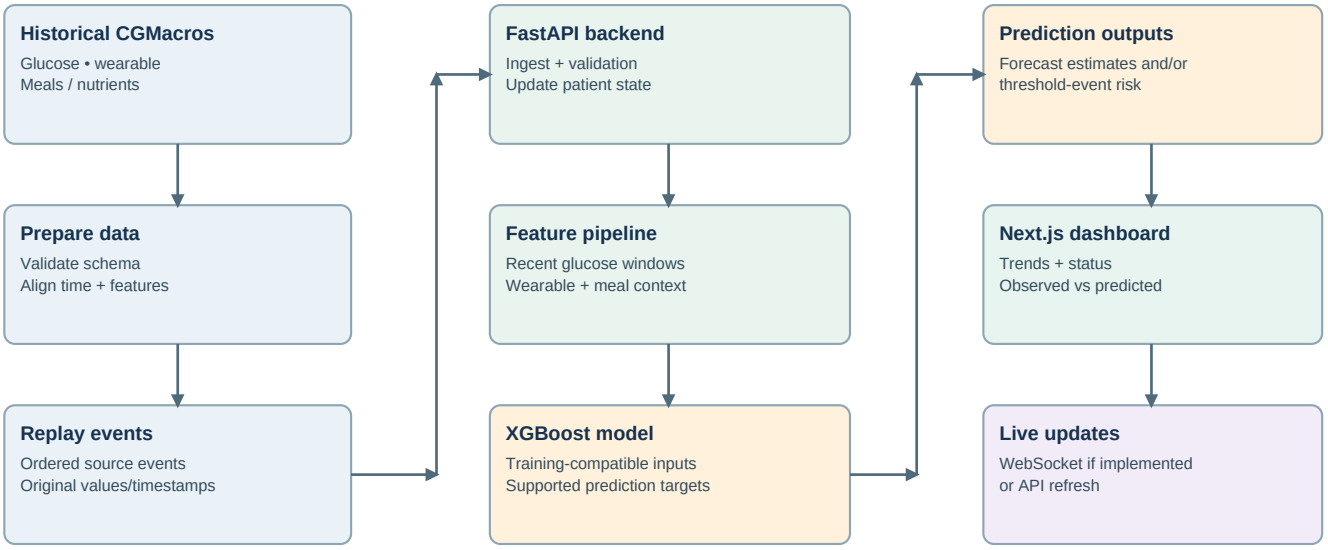
GLUCOSE TWIN

System Architecture & End-to-End Workflow



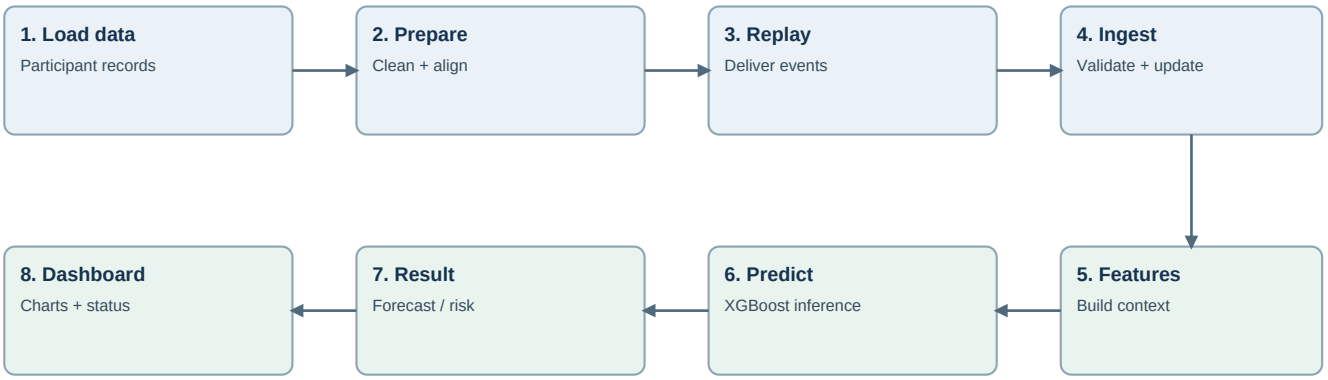
Reference diagram: verify routes, transport, and output types against the actual code.

Purpose: combine glucose, wearable and meal context with a trained XGBoost model, then show supported forecasts or threshold-risk outputs in the Next.js dashboard.

1. Component responsibilities

Layer	Responsibility	Flow
Historical data	Glucose, heart-rate/activity, meal and nutrient records.	CSV → observations
Preparation	Validate columns, align timestamps, handle missing fields, build features/events.	Raw → prepared data
Replay runner	Send historical events in order; playback speed controls delivery pace.	Event file → API
FastAPI	Validate requests, update patient/session state, coordinate inference.	Event → updated state
Feature pipeline	Build recent glucose windows and available wearable/meal context.	State → feature vector
XGBoost	Predict only the targets it was trained and evaluated for.	Features → estimate/risk
Next.js UI	Show trends, status, predictions and replay progress.	Results → dashboard
Live transport	WebSocket or API refresh, depending on current implementation.	Backend → browser

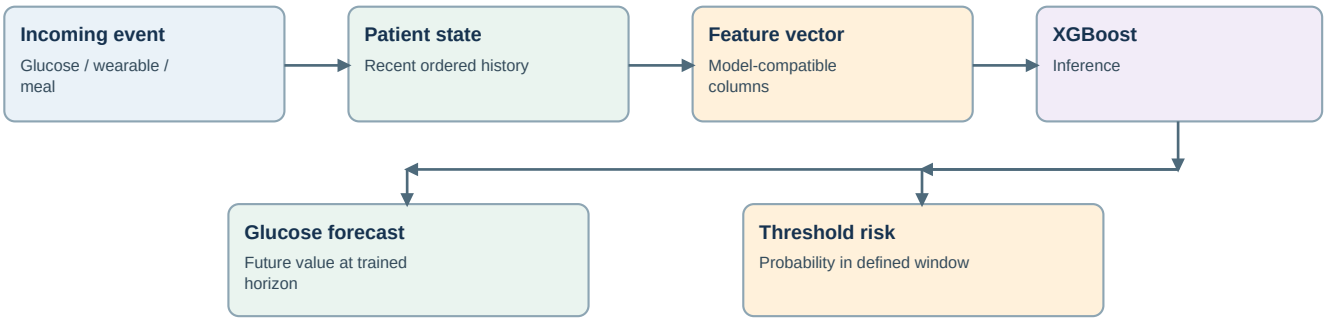
2. End-to-end workflow



Event time = source timestamp; delivery time = when the replay sends it.

The backend must be running at the same URL and port configured in the replay script. If not, requests fail before patient state or predictions update.

3. Prediction pipeline



Only display outputs actually trained, evaluated and returned.

Processing sequence

- **Receive:** accept a timestamped glucose, wearable or meal event.
- **Update:** validate the event and update the correct patient/session history.
- **Build features:** apply the same windows, units, missing-value rules and transformations used in training.
- **Predict:** run XGBoost with a model-compatible feature vector.
- **Present:** display observed readings separately from model estimates, with timestamps.

Output types are different

Output	Meaning
Glucose forecast (e.g. 30/60/120 min)	Estimated future glucose at a trained horizon.
Threshold risk in a window	Probability of a defined event during a specified time window.
Time to threshold crossing	Requires a validated time-to-event method or evaluated future trajectory; risk probability alone is not an exact time.

UI rule: do not label a probability as an exact predicted time. Display only the outputs actually supported by the model.

4. Replay timing and data integrity

Replay uses historical data to simulate an incoming stream. Keep source event time separate from the wall-clock delivery time.

Term	Meaning
Event time	Timestamp in the source record; used for ordering and time-aware features.
Delivery time	When the replay runner sends the event to the API.
Replay speed	Controls how quickly events arrive; must not change source values.

- Preserve source glucose/wearable values; do not fabricate intermediate readings to smooth charts.
- Use identical feature definitions and missing-value behavior during training and inference.
- Keep replay labelled as historical replay, not a fresh real-patient measurement.
- Show stale or missing data; do not silently treat missing values as zero.
- Version the model and feature schema where supported.

5. Runtime configuration

Example local configuration; verify the variable names in the repository:

```
GLUCOSE_TWIN_API_URL=http://127.0.0.1:8000
NEXT_PUBLIC_WS_URL=ws://127.0.0.1:8000
```

Start FastAPI first, then start replay in another terminal. For deployment, use HTTPS/WSS where applicable and keep secrets in environment variables.

6. Reliability and safety checks

- Validate payloads, timestamps, ordering and required fields.
- Monitor missing/stale glucose or wearable data and prediction freshness.
- Evaluate on held-out data and test training/inference feature parity.
- In-memory patient state can reset on restart and may not be shared across instances.
- This prototype is not a substitute for clinical advice and must not drive diagnosis, medication dosing or treatment decisions without appropriate validation.

Reference architecture based on the project context. Verify exact routes, persistence, live transport and prediction outputs against the current source code.